Final Project Design Document

Andrew Weckwerth, Luiz Blum

November 20, 2025

The final project is an HTTP/1.1 server that parses real protocols, implements HTTP operations, and supports multiple concurrent users safely and robustly.

Key Modules

We describe the key modules in the system, including the data structures and key units of computation.

Data structures: We create the following key data structures in the system:

- `Request`: A single HTTP/1.1 request data structure. (get or put)
- `Response`: An HTTP/1.1 response containing a status code and phrase. If using a GET request, the response will include the requested file's contents.
- `ThreadPool`: Data structure to store all the active threads.
- `LockMap`: Data structure to store all the reader's writer locks for each file.

Computational Modules

- `RequestHandler`: Processes a single GET or PUT request. Uses the RequestParser module and request and response data structures. Each active thread will use the RequestHandler module to process a request. Takes in a request data structure, returns a response to the client, and closes the connection.
- `RequestParser`: Parses the requestline and headerfield list. Used by the requestHandler.
- `ThreadPoolManager`: manages the thread pool by using the ThreadPool data structure and assigns requests to available threads.
- `main`: Starts the server, controls parameters like port and parallelism, establishes and manages the HTTP connections, and forwards requests to `ThreadPoolManager`.

Parallelism

Our HTTP server uses thread-based parallelism to execute concurrent requests. There is a threadPool that processes each request. It does not use `fork()` to create processes or use shared-memory to communicate. However, because they are threads, they all share the same memory, where all the server files are stored. At a high level, the parallelism in our HTTP server works through the following two components:

- Main Server Loop: This is the main loop that starts the server and accepts connections. It uses the `ThreadPoolManager` module to allocate requests to threads.

- Threads (Individual Requests): Inside of each thread, the RequestHandler module is used to process individual requests, generate responses, and close the connection.

Shared State

Since our HTTP server uses threads, the threads will share memory and synchronization is required to make sure the files are accessed safely. We will prevent data races by implementing a readers writers lock for each file. This way, concurrent requests to the same file are properly coordinated: multiple GETs can proceed in parallel, while PUT operations obtain exclusive access so that no reader ever observes a partially written file.

The readers writers lock ensures:

- Several concurrent GETs (reads)
- No reads during a write
- No writes during a write

Since each file requires its own reader/writer locks, we use a shared map that contains file-to-lock mappings. This will require its own single mutex lock, which will be briefly held by each thread to prevent corruption of the mappings.

The advantage of using a thread pool is that threads don't need to be created for each request, reducing overhead. Additionally, if we used processes for each request, context switches would be costlier and would require inter process communication when accessing shared files.

To test our server locally, we will use unit tests and also use curl to simulate real HTTP requests.